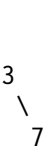
Principles of Programming Languages, 2025.01.20

Important notes
- Total available time: 2h (*multichance* students do not need to solve Exercise 3).
- You may use any written material you need, and write in English or in Italian.
- You cannot use electronic devices during the exam: every phone must be turned off and kept on your table.
- You cannot use library functions not covered in class in your code.

Exercise 1, Scheme (11 pts)

Consider a binary tree encoded as a list, where its first element is the current node, its second element contains the left sub-tree and the third element the right sub-tree. E.g. (1 (2 (4) (5)) (3 (6) (7))) encodes the tree:



Define a *tail recursive* procedure which takes a tree and returns the list obtained by traversing the tree *breadth first*. In the example, the resulting list must be (1 2 3 4 5 6 7)

Exercise 2, Haskell (11 pts)

Consider the following data definition:

```
data T a = A | B (T a) a | C a a deriving (Show, Eq)
```

1) Describe the data structure.

2) Make T an instance of Functor, Foldable, and Applicative.

Exercise 3, Erlang (11 pts)

We want to simulate a 6-side dice, where each face is implemented by a process running the following procedure, where X represents the value of the face:

```
dice_face(X) ->
    timer:sleep(rand:uniform(40)),
    receive
        stop -> bye;
        PID ->
            PID ! X,
            dice_face(X)
    end.
```

Write a procedure which simulates a dice: it first creates the processes for the faces, and waits for a value: the first one arriving in the message queue is the result of the roll. All the other messages must be purged from the queue, and the processes stopped.

Solutions

Ex 1

```
(define (tree2listTR T)
  (define (left-right T)
    (if (or (not (pair? T))(null? (cdr T)))
      '()
      (if (null? (cddr T))
        (list (cadr T))
        (list (cadr T)(caddr T)))))
  (define (helper Q R)
    (if (null? Q)
      (foldl cons '() R) ; reverse
      (let ((t (car Q)))
        (helper (append (cdr Q)
                        (left-right t))
                (cons (car t) R)))))  ; in reverse order, to avoid append
  (helper (list T) '()))
```

Ex 2

```
data T a = A | B (T a) a | C a a deriving (Show, Eq)

-- T is parametric sum type with a Null (A), a list-like cons (B), and a pair (C)

instance Functor T where
    fmap f A = A
    fmap f (B x y) = B (fmap f x) (f y)
    fmap f (C x y) = C (f x) (f y)

instance Foldable T where
    foldr f i A = i
    foldr f i (B x y) = f y (foldr f i x)
    foldr f i (C x y) = f x (f y i)

A +++ t = t
t +++ A = t
(B x y) +++ t = B (x +++ t) y
(C x y) +++ t = (B (B A y) x) +++ t

instance Applicative T where
    pure = B A
    A <*> x = A
    (B t f) <*> y = (fmap f y) +++ (t <*> y)
    (C f1 f2) <*> t = (fmap f1 t) +++ (fmap f2 t)
```

Ex 3

```
flush() ->
    receive
        _ -> flush()
    after
        0 -> true end.

dice() ->
    Ps = [spawn(fun () -> dice_face(X) end) || X <- [1,2,3,4,5,6]],
    [P ! self() || P <- Ps],
    receive
        V ->
            ok
    end,
    [P ! stop || P <- Ps],
    flush(),
    V.
```